ENGINEERING REVIEW · 2026-09-12

Full Platform Review & Hardening Report

A full-team pass over the app: every gate run, every journey walked, three P1 defects found and fixed with regression tests, and a clear map of what is solid, what is still risky, and what comes next.

Product · Engineering · Security · Data · AI · UX · QA · DevOps

KEJA AI · KEJA.APP

Table of Contents

1 Executive Summary

This report records a full-team review of Keja AI (keja.app)—product, engineering, security, data, AI, UX, QA and DevOps—performed on the live platform and the repository at commit 1eee9ae. The engagement walked the product as a real user in a real browser, ran every quality gate the pipeline defines, audited dependencies and secrets, and then fixed what the review found. Three product-grade defects were confirmed and repaired in this pass, each with regression tests: the platform's own advertised example search returned zero results; property detail pages reverted their SEO metadata after hydration; and the main listings page overflowed horizontally on phones.

The platform's foundation is stronger than its marketing suggests in one direction and weaker in another. It is stronger in honesty: the trust model—Trust Scores with evidence panels, a claims register, FACT/ESTIMATE labelling on every AI answer, and a governed analytics layer with redaction—is implemented with real discipline and test coverage. It is weaker in substance: Keja AI today is a 100% client-side static export. There is no server, no database in production, and no server-enforced authorization. Authentication is Google Sign-In verified client-side; roles, listings and two-factor state live in the browser's localStorage. This report treats that boundary as the platform's central engineering fact and labels every capability accordingly.

302	3	0	87	95
tests green (31 new this pass)	P1 defects fixed + regression-tested	secrets exposed; scan clean	live listings; 111 sitemap URLs	prerendered crawler pages

The quality pipeline is genuinely gated: typecheck, lint, unit tests, production build with artifact verification, deploy, and post-deploy smoke tests all run on every push to main, and the two commits from this review passed them end to end. The dependency audit found fourteen advisories—all in development toolchains (vitest, eslint, the Capacitor CLI); none ship in the production bundle. The single highest-leverage next step is unchanged from prior audits and is reaffirmed here with evidence: stand up the first server surface so authorization, persistence and the DeepSeek integration can move behind a trust boundary.

2 Current Product Status

Keja AI deploys as a Next.js 16 static export from gadda00/keja-ai main to Vercel, serving keja.app and the keja-ai-rho.vercel.app alias. The application is a hash-routed single-page app with 27 user-facing surfaces covering the full property lifecycle: discover, compare, verify, analyse, finance, invest, transact, tokenize and manage.

Surface	Status	Evidence
Deploy & CI	Live, gated	Typecheck, lint, 302 tests, artifact verification, smoke tests—all green on 1eee9ae
Property discovery	Working	87 listings (27 authored + 60 auto); map, filters, sort, save-search verified in browser
Natural-language search	Fixed this pass	Hero example '2BR Kilimani under 15M' returned 0 results; now returns 2 (regression-pinned)
Property detail + SEO	Fixed this pass	95 prerendered pages; hydrated title and JSON-LD now persist
Ask Keja AI	Working (deterministic)	Grounded, cited, FACT/ESTIMATE-labelled answers from a local corpus; no external LLM in production
Deal Analyst & calculators	Working	Mortgage, ROI, affordability—hand-computed values pinned by tests
Authentication	Client-side demo	Google Sign-In (GIS) verified in browser; session + role in localStorage—not server-enforced
Two-factor (TOTP)	Client-side demo	RFC 6238 correct, but enrolment + verification are device-local
Admin console	Client-side demo	Role + per-session 2FA gates in the UI only
PWA	Working	manifest, service worker, offline.html, installable on Android/iOS

The honest top-line: everything a visitor can see and do works and is tested; everything that requires a server (accounts, server-side authorization, persistent data, external AI calls) is either absent or runs as a clearly-labelled local simulation. The platform's own Trust Center documents several of these boundaries, which is the right instinct—this report extends that discipline to every surface.

3 What Works — Verified

Each item below was exercised on the live product or pinned by the automated suite during this review, not taken from documentation.

- Discovery journey: hero, filters, sort, map view, save-search with alerts—walked in a headless browser against keja.app with zero console errors.
- Deep links: /properties/KJA-001 and 94 sibling routes serve prerendered static HTML with correct titles, canonicals, OG tags and JSON-LD (verified via curl).
- Ask Keja: real questions answered with citations, confidence labels and honest abstention; escalation policy covered by 12 test cases plus false-positive guards.
- Financial math: mortgage annuity, amortisation, extra payments, ROI and DTI caps—hand-computed expectations locked in tests.

- Trust system: 12-factor Trust Score (weights sum to 1, bounds tested), 24-claim register with disclosure rules, 90-day evidence freshness with boundary-day tests.
- Data integrity: every listing passes schema validation and every image path resolves to a real file on disk—the suite fails the build otherwise.
- Security headers on keja.app: CSP, HSTS, X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy, COOP same-origin-allow-popups—all verified live.
- robots.txt, sitemap.xml (111 URLs) and RFC 9116 security.txt serve correctly; admin/account/dashboard routes are disallowed.
- Governed analytics: schema-validated event envelopes with PII redaction, unknown-key stripping, ring-buffer caps and corruption recovery—all tested.
- Accessibility basics: skip link, semantic headings, labelled form controls, aria-pressed toggles—confirmed in the accessibility tree of every page visited.

4 What Is Simulated, Partial or Absent

Stated plainly, because users and investors will eventually ask. Nothing in this section is claimed to be production-ready.

Capability	Classification	What it really is today
Accounts & sessions	2—Demo	Google ID token decoded and verified client-side; the session, role and registration state live in localStorage on one device only
Admin authorization	2—Demo	UI gates (role allowlist + 2FA step); no server exists to enforce them—anyone inspecting the bundle knows the admin surface exists
2FA (Google Authenticator)	2—Demo	Correct RFC 6238 TOTP, but the enrolment secret and verification are device-local—it protects a local session, not an account
Marketplace listings (60 of 87)	3—Partial (auto)	Auto-Pilot generated inventory from an internal generator—schema-validated and quality-scored, but not real inventory
Authored listings (27)	1—Implemented	Hand-authored sample inventory with evidence records—illustrative, not live partner stock
Transaction desk, tokenization	4—Guide only	Educational flows describing how the future process will work; no funds move, no contracts execute
DeepSeek integration	5—Scaffolded	Complete server-only adapter with redaction, budgets, JSON schema validation and policy review—dormant because a static export has no server to arm it
Prisma schema (18 models)	5—Scaffolded	Designed Phase-2 backend: users, properties, evidence, tenancy, payments, claims, audit—unused in production today

Capability	Classification	What it really is today
Server-side analytics sink	5—Planned	Events are generated, validated and stored locally; no remote collection endpoint is configured

The classification numbers follow the review's five-level scale: 1 fully implemented, 2 demo or simulated, 3 partially implemented, 4 broken (nothing in production currently earns this after this pass's fixes), 5 recommended future functionality. The distinction that matters most for trust marketing: the verification and scoring claims on authored listings are real computation over declared evidence, while the auto-listings are labelled as generated in their detail views. That honesty is a feature; keep it.

5 Critical Problems Found and Fixed

Three defects with direct revenue relevance were confirmed on the live product and repaired in commit 1eee9ae, each carrying new regression tests. They are documented here with root cause, fix and verification, because they illustrate how the platform's funnel can silently fail while every gate stays green.

P1-1 — The advertised search example returned zero results

The home-page hero invites the exact query “2BR Kilimani under 15M”. The results page matched the whole string as a literal substring against listing text, so the product's own example returned “0 of 87 listings”—while KJA-A0162, a 2-bedroom Kilimani apartment for KES 10.1M, sat in inventory. Every user who trusted the placeholder hit an empty state. The fix introduces src/lib/queryParser.ts: the free-text query is parsed into structured intent—bedroom count (“2BR”, “two-bedroom”), price ceiling (“under 15M”, “below 150k”, absolute KES), purpose, property type and known area names—and leftover tokens match with AND semantics. The same matcher now also drives saved-search alerts, so alerts can never diverge from what the results page shows. Verified live: the hero query now returns the two qualifying Kilimani listings. 25 tests pin the parser, including a real-inventory regression that fails if the hero example ever returns empty again.

P1-2 — Property detail pages reverted their SEO after hydration

The prerender pipeline emits correct per-listing titles, descriptions, OG tags and JSON-LD into static HTML—crawlers saw the right thing. But the moment JavaScript hydrated, a comment in KejaApp.tsx claimed detail views “set richer entity-specific meta … so their tags win”—and no detail view had ever called usePageMeta. The route-level fallback reverted the tab title to “Keja AI” and stripped the structured data that Google renders. The fix creates src/lib/detailMeta.ts as the single derivation of listing, article and area-guide metadata, used by both the live views (post-hydration) and the prerender script (crawler-facing) so the two can never drift. Verified live: the hydrated title now reads “Elegant 2-Bedroom Residence in Kilimani—Kilimani, Nairobi · Keja AI” and the RealEstateListing JSON-LD persists after hydration. 6 parity tests pin the derivation.

P1-3 — Mobile horizontal overflow on the listings page

At a 390px viewport the properties page overflowed horizontally by 67px, producing the worst mobile symptom a listings page can have: the whole page jiggles sideways while scrolling. Root cause: the header actions row (Map view, Save search, and a fixed w-44 sort select) totalled 441px with no wrap. The row now wraps and the sort control flexes to the available width below the sm breakpoint. All 27 routes were re-scanned at 390px: zero overflow anywhere.

6 Security Review (OWASP-aligned)

The review examined authentication, session handling, authorization, secret custody, input validation, headers, transport security and data exposure—with a strict eye on the difference between client-side demo security and server-enforced security.

Findings register

ID	Sev	Finding	Status / position
SEC-A	Critical (design)	No server exists: authorization, sessions and roles are browser-state; the admin surface is reachable in the shipped bundle	Known architectural boundary—the Phase-2 backend is the fix; nothing in the current threat model pretends otherwise
SEC-B	High (accepted)	Google ID token verified client-side only; a crafted client can skip verification and mint a local 'admin' session	Mitigated by honesty: no server-side action trusts it; documented in Trust Center; resolves with the backend
SEC-C	Low	CSP allows 'unsafe-inline' scripts (Next.js static export requirement)	Acceptable today; hash-based CSP is the post-backend improvement

ID	Sev	Finding	Status / position
SEC-D	Low	14 dependency advisories in dev toolchains (vitest, eslint, capacitor CLI chains incl. a uuid buffer issue)	None ship in the production bundle; upgrade @capacitor/cli at next mobile release
SEC-E	Info	Secret hygiene verified: secret scan of tree and history clean; .env never committed; DEEPSEEK key path is server-guarded with a build-time tripwire	Good state; keep the pre-push scan
SEC-F	Info	Headers strong: CSP, HSTS 1y, XFO DENY, nosniff, Referrer-Policy, Permissions-Policy, COOP tuned for Google popups	Verified live on keja.app

The one incident this session: a scripted over-broad 'git add' briefly staged .env and build artifacts into a local commit. It was caught before push (remote never received it), the file contained only a local SQLite path—no credential—and the commit was rebuilt cleanly. It is recorded here because process transparency is a security control: the corrected workflow stages explicit paths only, and the pre-push secret scan remains the backstop. The structural conclusion stands: no client-side gate in this codebase should ever be described as server-side authorization, and this report does not describe them that way.

7 UX and Journey Findings

Twenty journeys were reviewed from the accessibility tree and live behaviour. The shell is disciplined—consistent typography, a working skip link, labelled controls, honest empty states with recovery actions, and helpful placeholder examples. The defects below were the exceptions, plus friction worth tracking.

- Search feedback (fixed): the hero example empty state was well-designed—and pointed at the P1-1 bug; the fix makes the empty state rare and truthful.
- Mobile layout (fixed): the only overflow on any route was the listings actions row; re-scan clean.
- AI intent precision: Ask Keja answered a deposit-check question with financing fundamentals—grounded and labelled, but intent routing is keyword-adjacent; worth improving once a real provider is armed.
- Loading states: skeleton components shipped this cycle after remediation, but no view is async yet—they become valuable the moment the backend arrives.
- Trust presentation: evidence panels, claim disclosures and the 'why this answer' affordance are genuinely strong differentiators—keep them prominent in any redesign.
- Onboarding: the account section's group-selection registration flow (renter/landlord/developer/agent/investor) is clear; consider surfacing it pre-authentication as a value

preview.

8 Technical Review

The codebase is unusually tidy for its size: strict TypeScript with zero ignored errors, React Compiler lint rules enforced, zod boundaries at every persistence read, RFC-correct crypto utilities, and a test suite that pins hand-computed financial mathematics.

Area	Assessment	Evidence
Type quality	Strong	tsc --noEmit clean; ignoreBuildErrors: false; no 'any' escapes found in reviewed modules
Boundaries	Strong	zod schemas validate auth, listings, tokenize and auto-listing payloads; invalid entries drop loudly (tested)
Component architecture	Good	27 route views + shell; error boundary wraps the app; lazy view loading with a labelled status region
Bundle discipline	Good	Static export, immutable hashed assets, WebP srcset images (60% smaller), 95 prerendered pages
Observability	Partial	Cookieless governed events locally + hourly production smoke checks; no remote sink yet
Dead code	Minor	The performance library added last cycle was unused by any view (now lint-clean and tested, but still unwired)—wire it when async views arrive
CI/CD	Strong	Gated typecheck, lint, tests, artifact verification, deploy, post-deploy smoke; two green runs this session

9 Data and Database Review

The Prisma schema is a credible 18-model domain design—users, sessions, properties, evidence, tenancy, rent payments with double-entry guards, claims, KYC records and audit entries with sensible cascades and opaque tokens. It is entirely unused in production: the static export has no server to run it.

Today's data reality: 27 authored listings with evidence records, 60 Auto-Pilot generated listings (schema-validated, images verified on disk by the test suite), a neighbourhood gazetteer, 6 long-form insight articles and the user's browser-stored state. Every dataset carries its provenance in code, and the Trust Center discloses the verification basis of each claim tier—this is the right foundation for the data dictionary the backend phase must formalize. The migration path is already built: the API seam (api/client.ts) rejects

cleanly while unconfigured, and the store-twin hook lets domains move from localStorage to REST one line at a time. The review recommends activating exactly that path, domain by domain, starting with authentication and listing submissions—the two datasets with real integrity requirements.

10 AI and the DeepSeek Path

The intelligence layer is the platform's most future-ready asset. Every AI surface routes through one governed gateway: classify (policy catalogue decides regulated questions upfront), redact (phones, IDs, emails stripped before any text can leave), retrieve (authorization-first corpus search with sufficiency gates), generate (provider seam), review (prohibited-advice and citation checks) and audit (governed events for every outcome).

Today the generate step is a deterministic local engine—honest, cited, and incapable of inventing ownership or title claims. The DeepSeek adapter is complete and battle-designed but dormant: it refuses to run in a browser (a build-time tripwire enforces server-only), requires DEEPSEEK_ENABLED plus a key in a server environment, applies a 12-second timeout, one bounded retry, token budgets, strict JSON outputs re-validated application-side, and abstains rather than leaking raw model text. Because production is a static export, arming it requires the first server surface—precisely the Phase-2 backend. The activation checklist, once the endpoint exists: set the key as a Vercel environment secret (never NEXT_PUBLIC), set DEEPSEEK_ENABLED=true, run the small controlled test conversation from the runbook, then swap the provider behind the existing seam. Cost control is already in the adapter via input and output token caps; add per-user rate limits at the endpoint.

> *The AI design principle that must survive every upgrade: the system never confidently invents ownership, title status, valuations, loan approval or returns—and it says so in the UI.*

11 RAG and Property Intelligence Plan

The retrieval half of RAG already exists in miniature: an approved public corpus (listings, area insights, policy documents) with chunking, area boosting, sufficiency thresholds and citation refs that the UI renders as source chips. The escalation policy (12 categories) functions as the abstention layer.

The backend phase scales this into real property intelligence. Ingestion: property records, verification evidence, area guides and partner documents enter with source metadata, versioning and freshness stamps. Embeddings land in a vector store beside the operational database; retrieval stays authorization-first so no private document crosses a user boundary. Re-ranking and citation continue through the existing gateway contract. The golden evaluation set should start from the 87 current listings and the questions users actually

asked this quarter (the governed event store has them, minus PII)—measure retrieval recall, citation correctness, groundedness, abstention quality, latency and cost before any provider swap, and re-run the suite on every corpus or model change.

12 Analytics and Metrics Plan

The event taxonomy is implemented and governed: 30+ event types across search, viewing, saving, comparison, listing submission, AI questions and answer feedback, calculator use, finance interaction, partner referral and admin review—each schema-validated, PII-redacted, ring-buffered locally and emitted through a single envelope contract. What is missing is the destination.

The recommendation: keep the local mirror as the offline buffer, and add a first-class server sink when the backend lands—the envelope already carries everything a warehouse needs. Metrics worth the pipeline: activation (first search to first property view), search quality (zero-result rate by parsed intent—the new queryParser makes this measurable for the first time), trust engagement (evidence-panel opens per listing view), conversion (saved search to contact request to listing submission), marketplace liquidity (new listings per week by source), and AI quality (answer feedback rate, abstention rate, escalation rate). Each is decision-bearing; none is vanity.

13 Performance Plan

Current state is healthy for a static export: WebP with responsive srcsets, immutable hashed assets, prerendered crawler pages, and an hourly availability check. The mobile overflow fix this session removed the one layout defect the route scan could find.

- Next (P2): wire the now-tested skeleton and progressive-image library into the first async views when the backend lands—perceived-load work is already built.
- Next (P2): add a Lighthouse budget to CI once server surfaces exist; today the static shell scores well but there is no regression alarm.
- Watch: the auto-listings dataset grows weekly—the sitemap and prerender scale linearly and stay fast, but revisit bundle size when it passes ~150 listings.
- Keep: the per-asset cache policy in vercel.json (immutable static, 7-day media, must-revalidate sw.js)—verified live.

14 Testing and Quality

The suite now stands at 302 tests across 22 files, all green, all run as a CI gate on every push. This session added 31 of them. The most valuable pattern in the suite is what this review leaned on repeatedly: real-data regressions—the parser test fails if the hero query stops matching real inventory; the data-integrity test fails if any listing image goes missing.

New suite file	Covers	Tests
tests/queryParser.test.ts	Intent parsing (beds, price units, purpose, type, area), AND-token semantics, price-on-application, real-inventory regression on the hero example	25
tests/detailMeta.test.ts	Shared entity-meta derivation: titles, descriptions, JSON-LD types, rental / month labels, POA listings, tagline fallback	6

Gaps that matter, honestly ranked: no end-to-end browser suite in CI (this review used a headless browser manually—the three defects it found would all have been caught by one scripted journey through the hero search, a detail page and a mobile viewport); no visual regression; and role-restriction tests exist at the store level but not against a future server. The single highest-leverage addition is a three-journey Playwright run in CI: home search, detail hydration title, mobile overflow—each defect class this session caught, permanently guarded.

15 Prioritized Roadmap

Ranked by leverage, with the same P0-P3 scale the review used. Items marked DONE shipped in commit 1eee9ae this session.

P	Item	Why / acceptance criteria
DONE	Natural-language search (P1)	Hero example returns qualifying listings; regression-pinned against real inventory
DONE	Hydrated detail SEO (P1)	Entity titles and JSON-LD persist post-hydration; prerender and view share one derivation
DONE	Mobile overflow (P1)	Zero horizontal overflow on all 27 routes at 390px
P0	First server surface	Vercel function + Prisma: server-verified sessions, role enforcement, listing submission persistence—the SEC-A/B resolution
P0	Move TOTP + admin gates server-side	Enrolment secrets and verification behind the endpoint; the client demo retires

P	Item	Why / acceptance criteria
P1	Three-journey Playwright suite in CI	Hero search, detail hydration, mobile viewport—guards every defect class this review found
P1	Arm DeepSeek behind the seam	Key as server secret; controlled test; metrics: groundedness, abstention rate, cost per answer
P1	Analytics sink + dashboards	Ship the governed envelopes to the warehouse; the activation and search-quality metrics light up
P2	RAG scale-up	Embed the corpus; golden eval set from real queries; citation and groundedness gates in CI
P2	Wire the performance library	Skeletons + progressive images on the first async views
P3	Kiswahili UI depth, Lighthouse budget, hash-based CSP	Polish layer once the server boundary exists

16 Changed Files, Tests, Remaining Risks

Commit 1eee9ae (10 files, +666/-70): new src/lib/queryParser.ts and src/lib/detailMeta.ts with their test suites; PropertiesView, PropertyDetailView, ArticleDetailView, AreaGuideView, searchStore and the prerender script updated. Commit a8edc4f earlier in the session (14 files) repaired the broken performance library that had failed three consecutive deploys.

Quality gates executed and recorded: typecheck clean; eslint clean; 302/302 tests across 22 files; full production build with artifact verification PASSED (95 prerendered pages, 111 sitemap URLs); dependency audit triaged to dev-tool chains only; secret scan clean across tree and history; live smoke of headers, security.txt, robots and sitemap on keja.app; and browser-level verification of the three fixes on the served build.

Remaining risks — stated without decoration

The platform's central risk is architectural: until the backend exists, accounts, roles, admin access and 2FA are browser-state, and the marketplace's authoritative data lives in the repo and the user's localStorage. No amount of client hardening changes that—only the P0 server work does. Secondary risks: the single maintainer's GitHub PAT remains un-rotated after its uses in these sessions (rotate it); the auto-listing generator grows inventory that is honest but synthetic—cap its share of the marketplace as real supply arrives; and CI has no end-to-end browser gate, so the defect class this review caught in the browser can regress silently. Finally, Google OAuth's client secret custody was configured via Vercel environment variables—verify rotation access to that console remains available.

17 Recommended Next Three Actions

If only three things happen next, these three.

- 1. Approve and build the first server surface (Vercel functions + Prisma): sessions, roles and listing-submission persistence behind a real trust boundary. Everything else unlocks from here—2FA, admin enforcement, DeepSeek, the analytics sink.
- 2. Add the three-journey Playwright suite to CI (hero search returns results; detail title survives hydration; zero mobile overflow). This session proved those three checks catch the defect classes that all other gates miss.
- 3. Rotate the GitHub PAT and re-verify Vercel/Google console access—five minutes, closes the only credential-age risk the review found.

The platform this review leaves behind is honest about what it is, genuinely tested for what it claims, and now free of the three defects that were quietly taxing its funnel. It is not yet production-grade infrastructure for accounts and transactions—and it no longer claims to be. The path from here is short, concrete and already designed into the codebase.